

5. Nonlinear linear regression

Frank Edwards

Returning to the height model

The anatomy of a linear model

Likelihood: $h_i \sim \text{Normal}(\mu_i, \sigma)$

Linear model: $\mu_i = \alpha + \beta x_i$

Prior: $\alpha \sim \text{Normal}(150, 25)$

Prior: $\beta \sim \text{Uniform}(0, 5)$

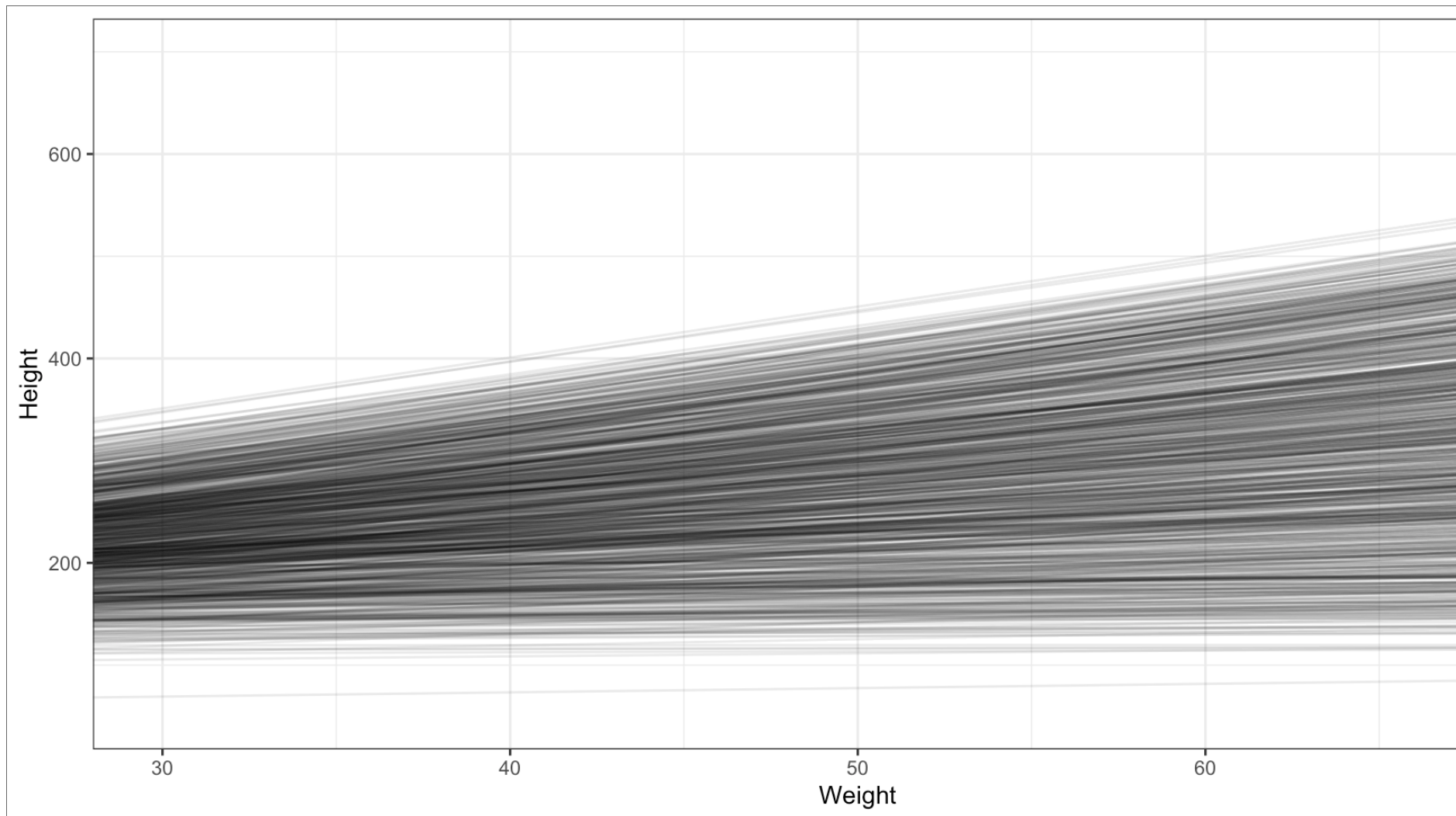
Prior: $\sigma \sim \text{Uniform}(0, 10)$

Estimating this model

```
1 data(Howell1)
2 d<-Howell1 |>
3   filter(age>=18)
4 m0<-quap(
5   flist = alist(
6     height ~ dnorm(mu, sigma),
7     mu<-a + b * weight,
8     a ~ dnorm(150, 25),
9     b ~ dunif(0, 5),
10    sigma ~ dunif(0,10)
11  ),
12  data = d
13 )
```

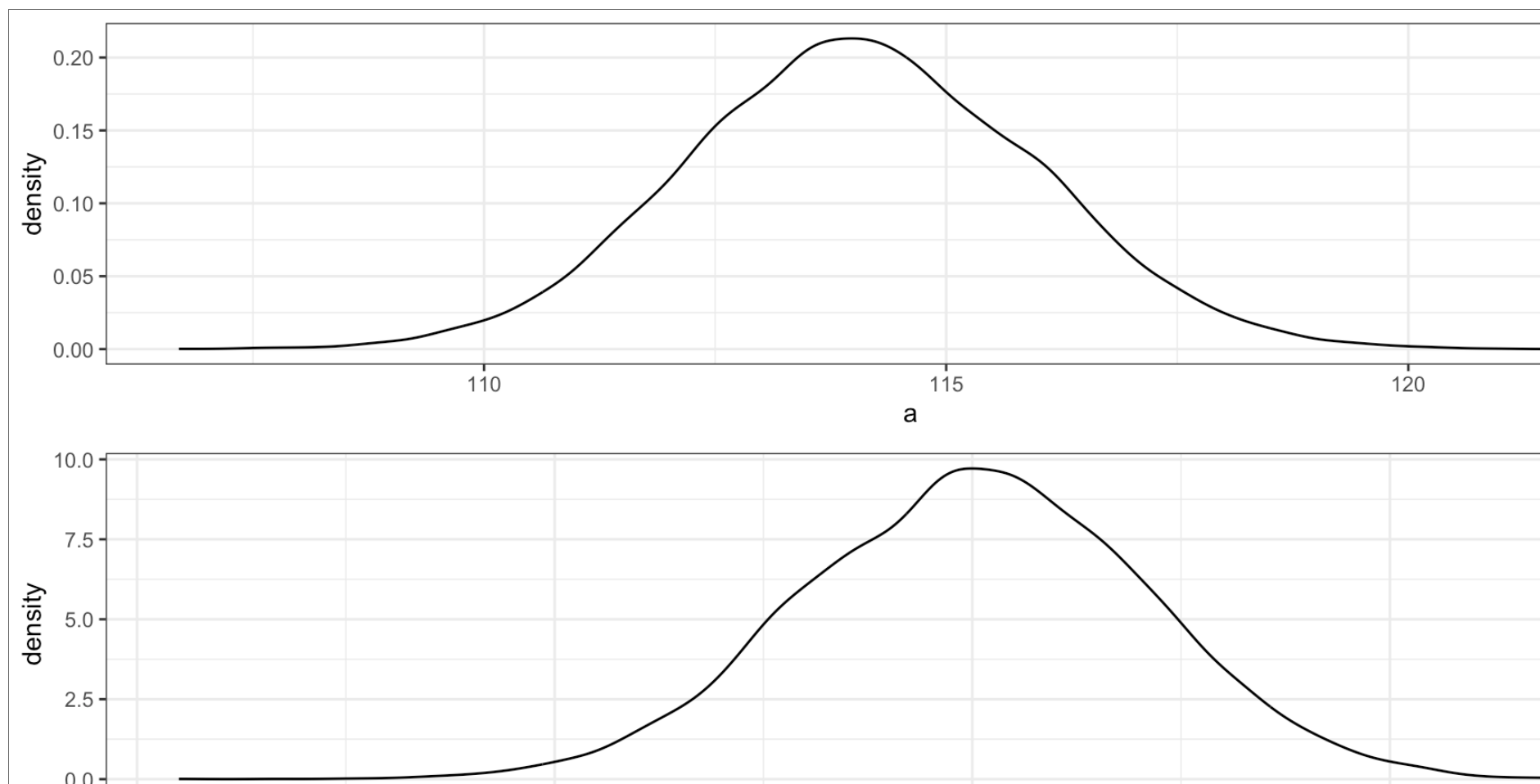
Generate prior predictions

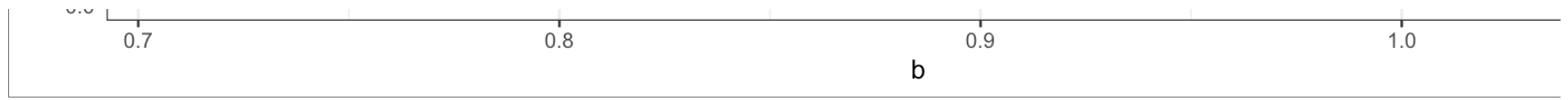
Prior predictions let us confirm that our priors make logical sense for our question



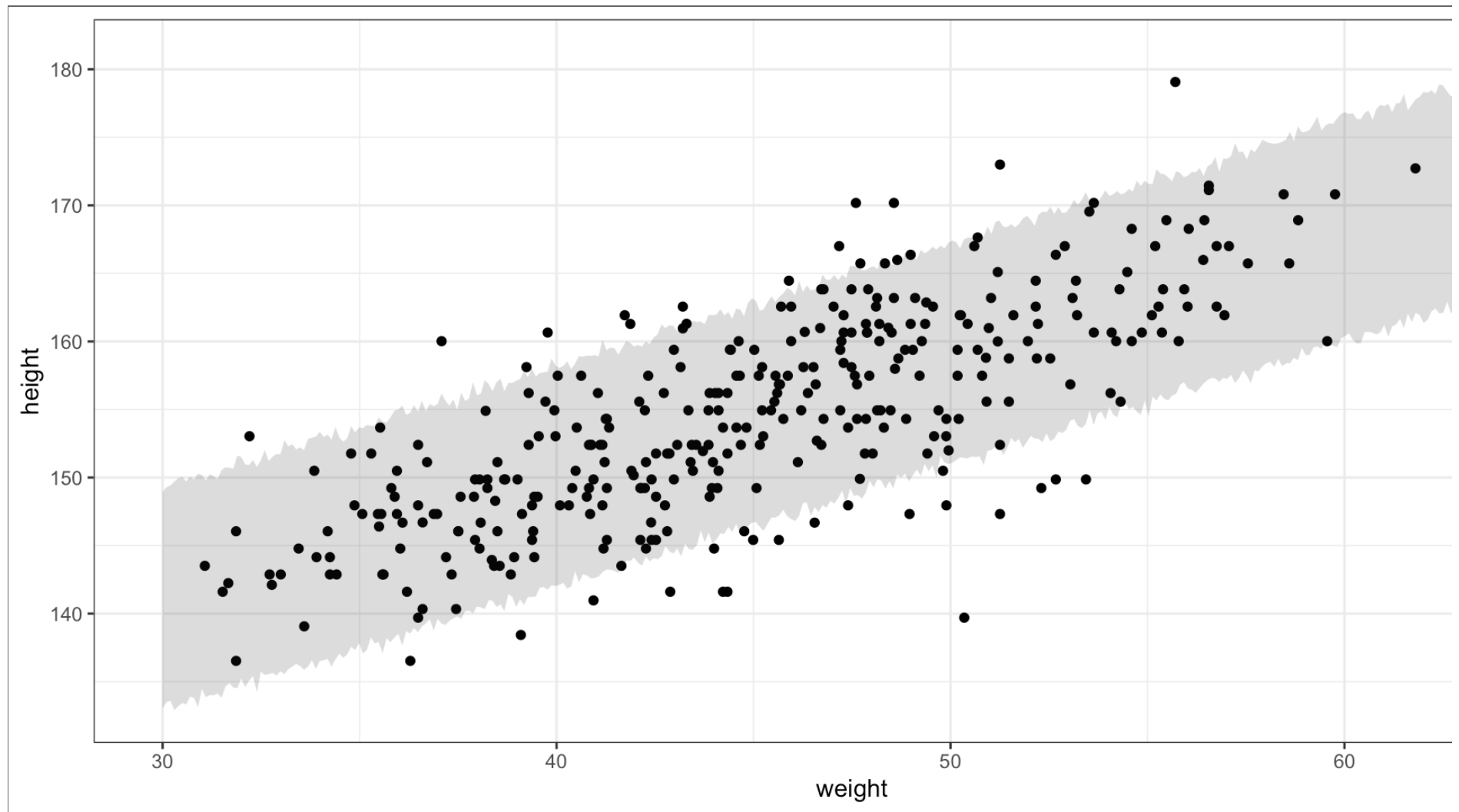
Draw posterior samples and visualize parameters

	mean	sd	5.5%	94.5%
a	114.0805099	1.90021345	111.0436018	117.1174180
b	0.9006544	0.04181117	0.8338321	0.9674768
sigma	5.0719670	0.19116410	4.7664498	5.3774842



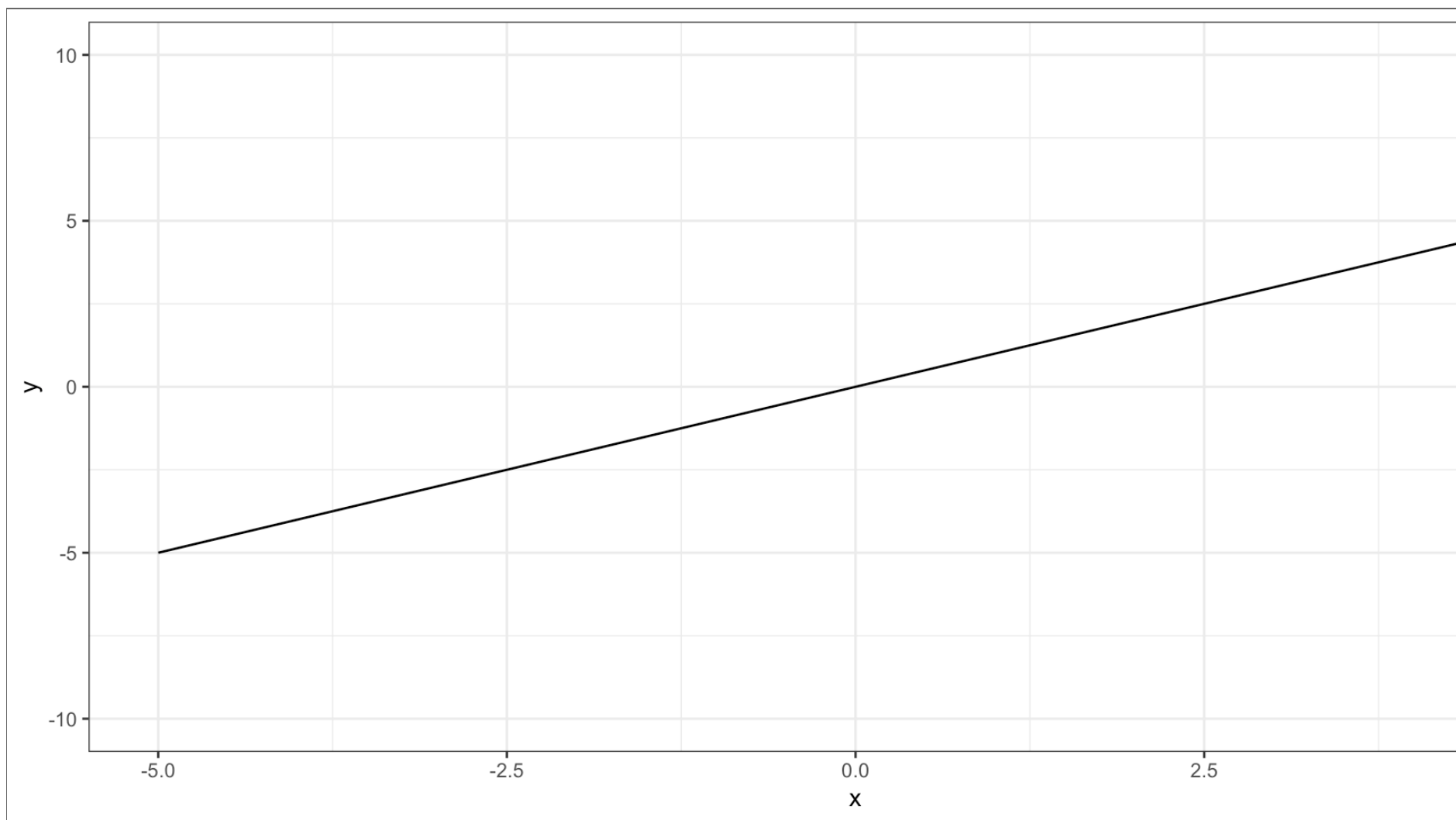


Predict from the posterior and compare to observed

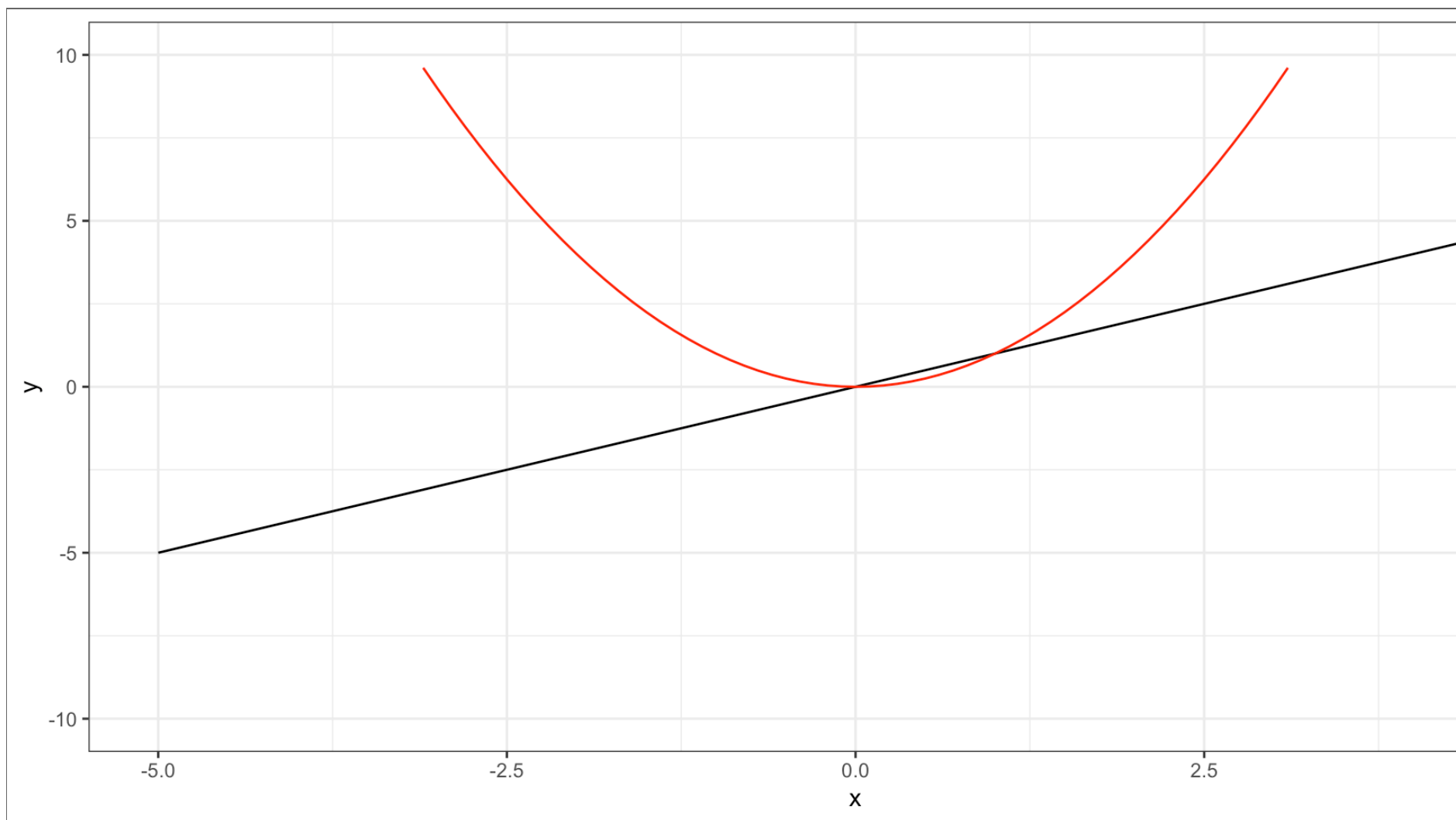


Fitting curves in linear regression models

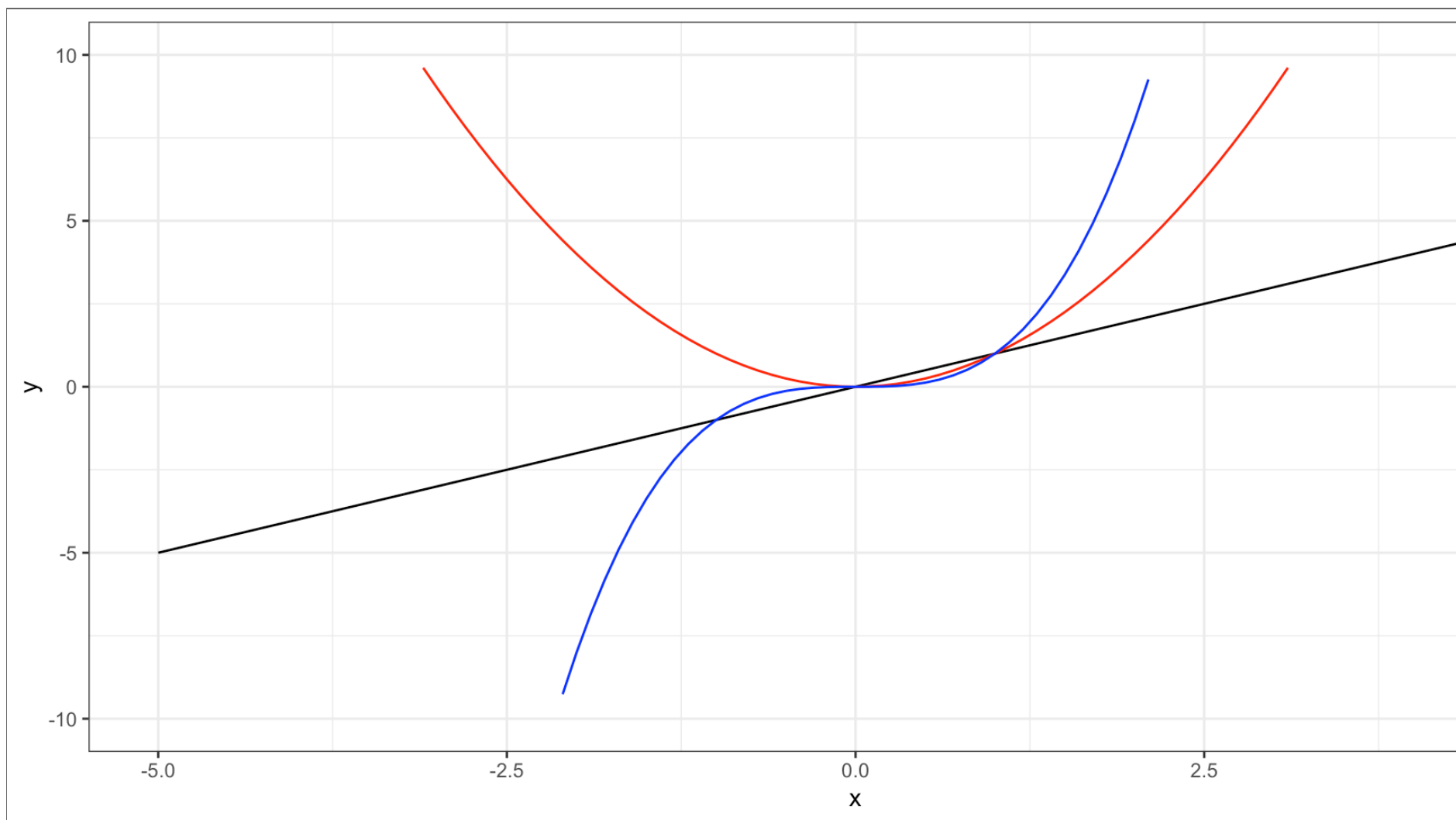
Polynomials: linear



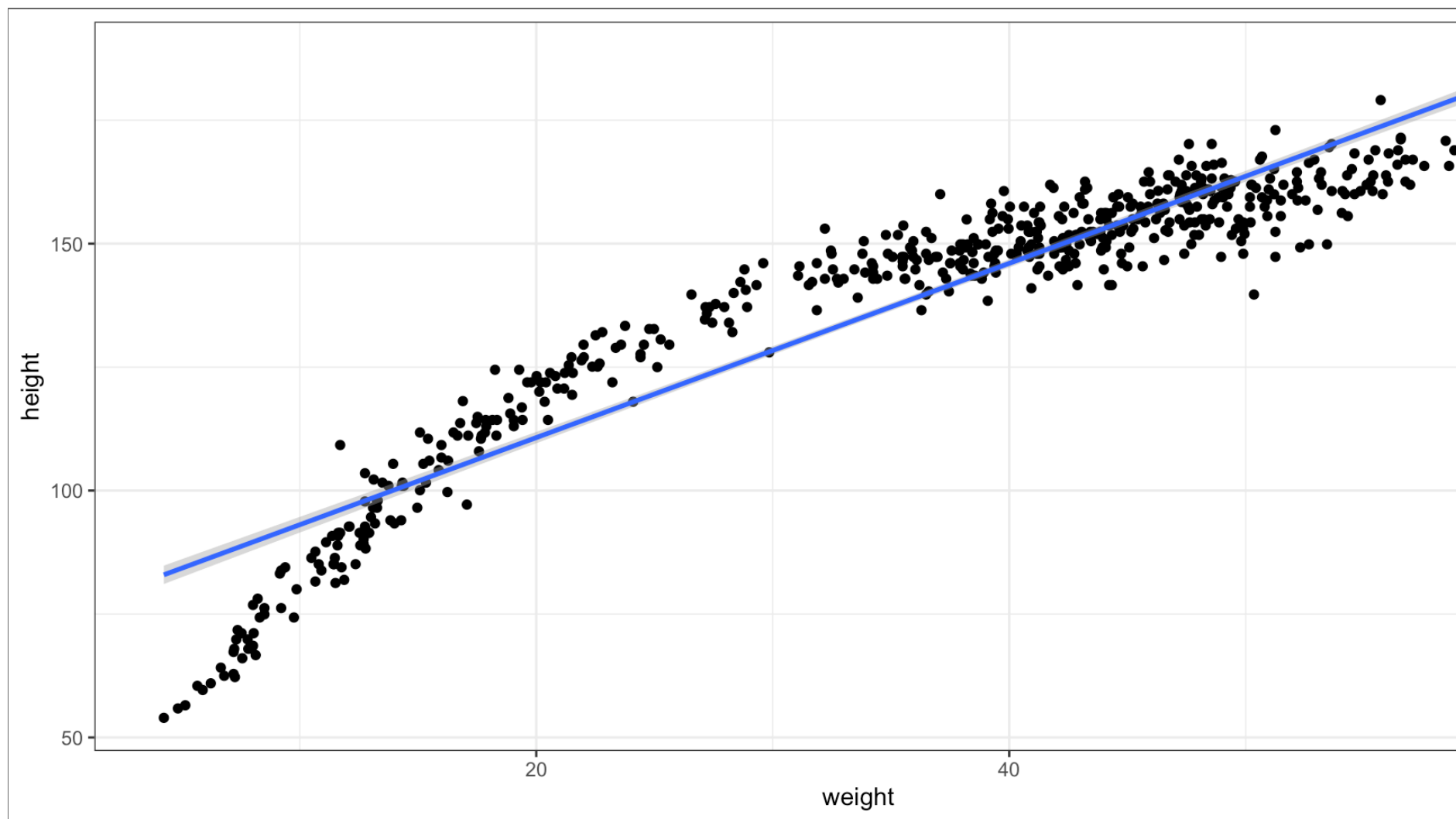
Polynomials: quadratic



Polynomials: cubic



When do we want a polynomial?



Defining a model with a polynomial term

$$h_i \sim \text{Normal}(\mu, \sigma)$$

$$\mu = \alpha + \beta_1 x_i + \beta_2 x_i^2$$

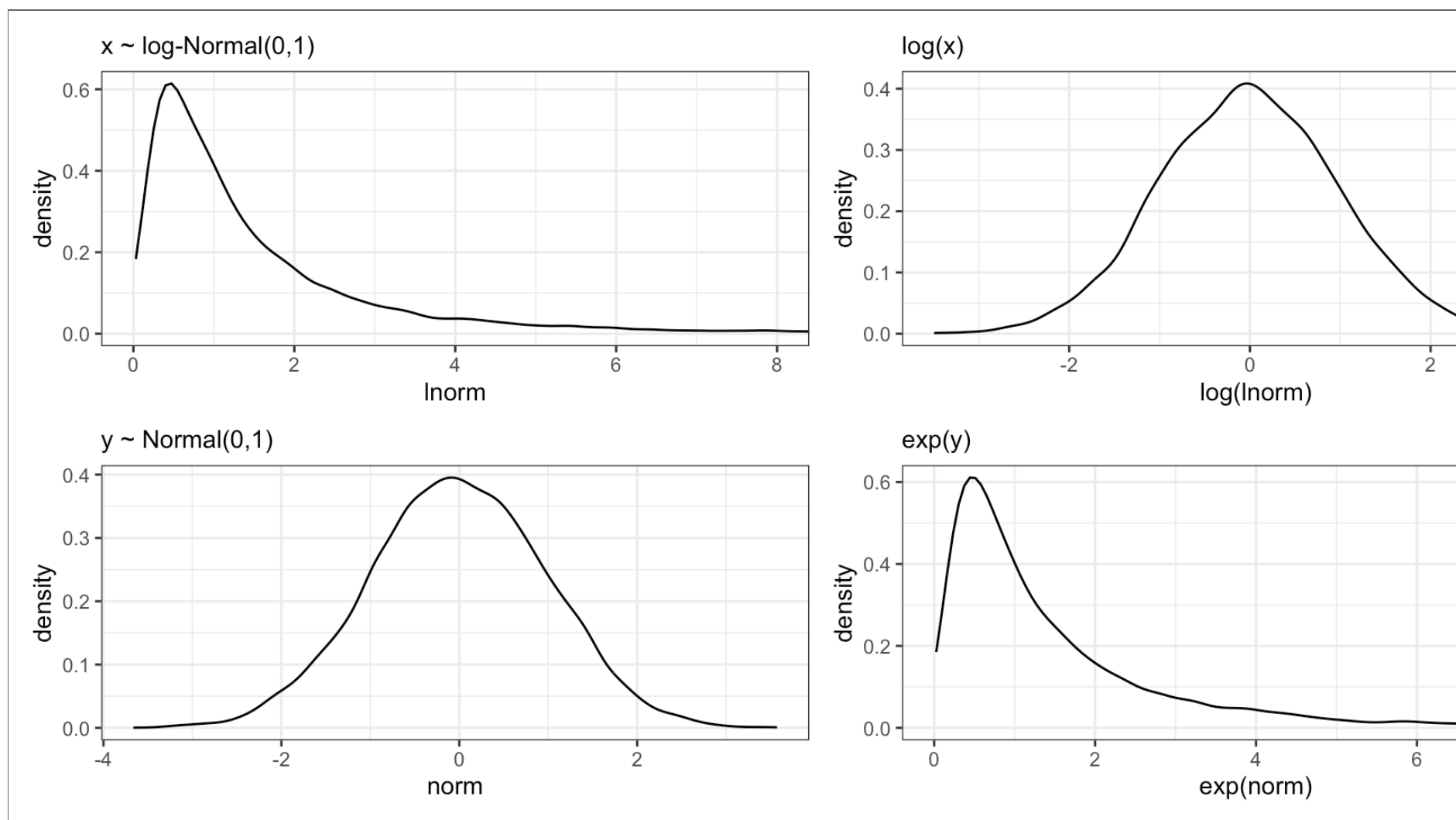
$$\alpha \sim \text{Normal}(0, 5)$$

$$\beta_1 \sim \text{log-Normal}(0, 1)$$

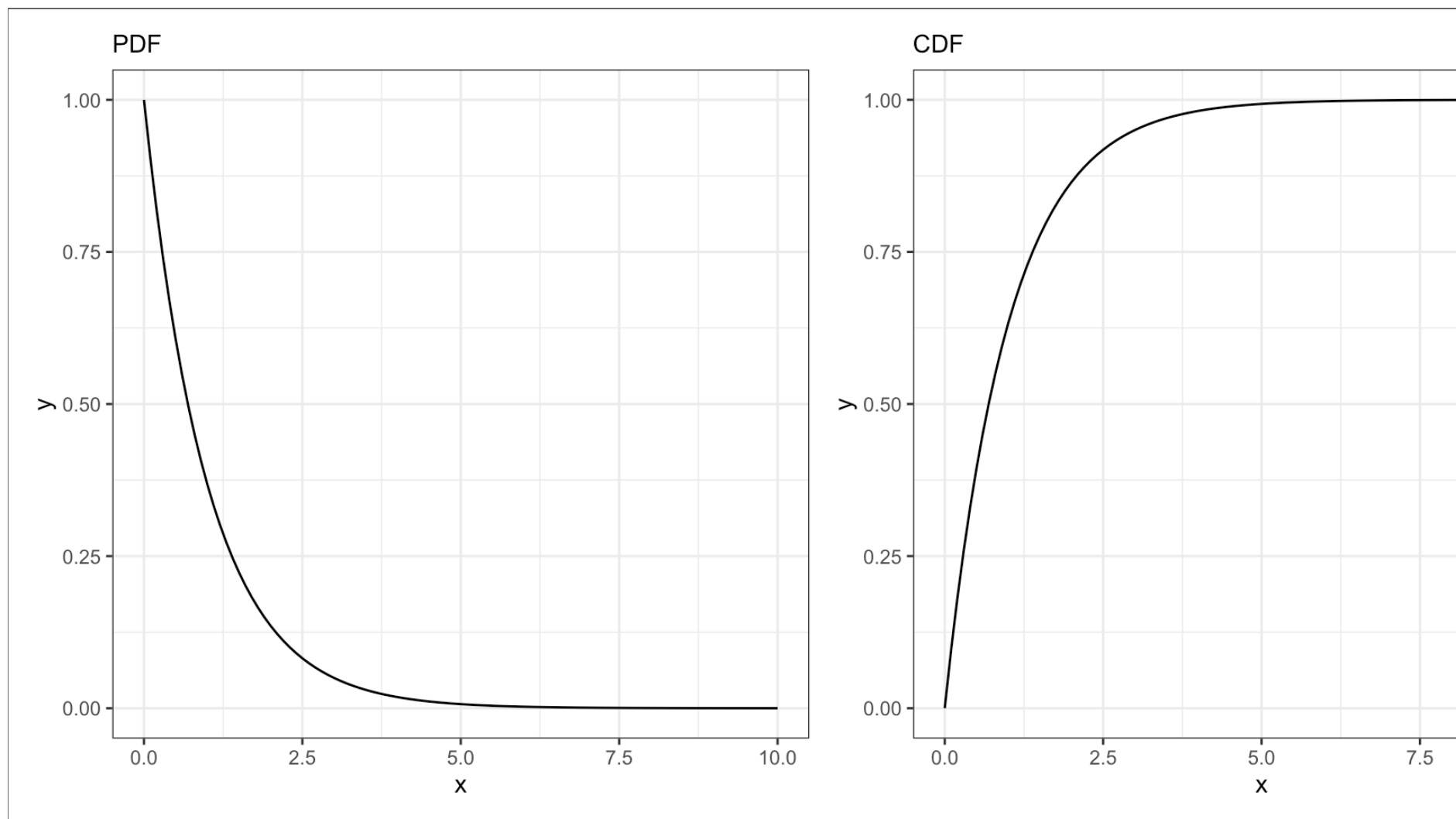
$$\beta_2 \sim \text{Normal}(0, 1)$$

$$\sigma \sim \text{Exponential}(1)$$

Wait... what is a log-Normal distribution??



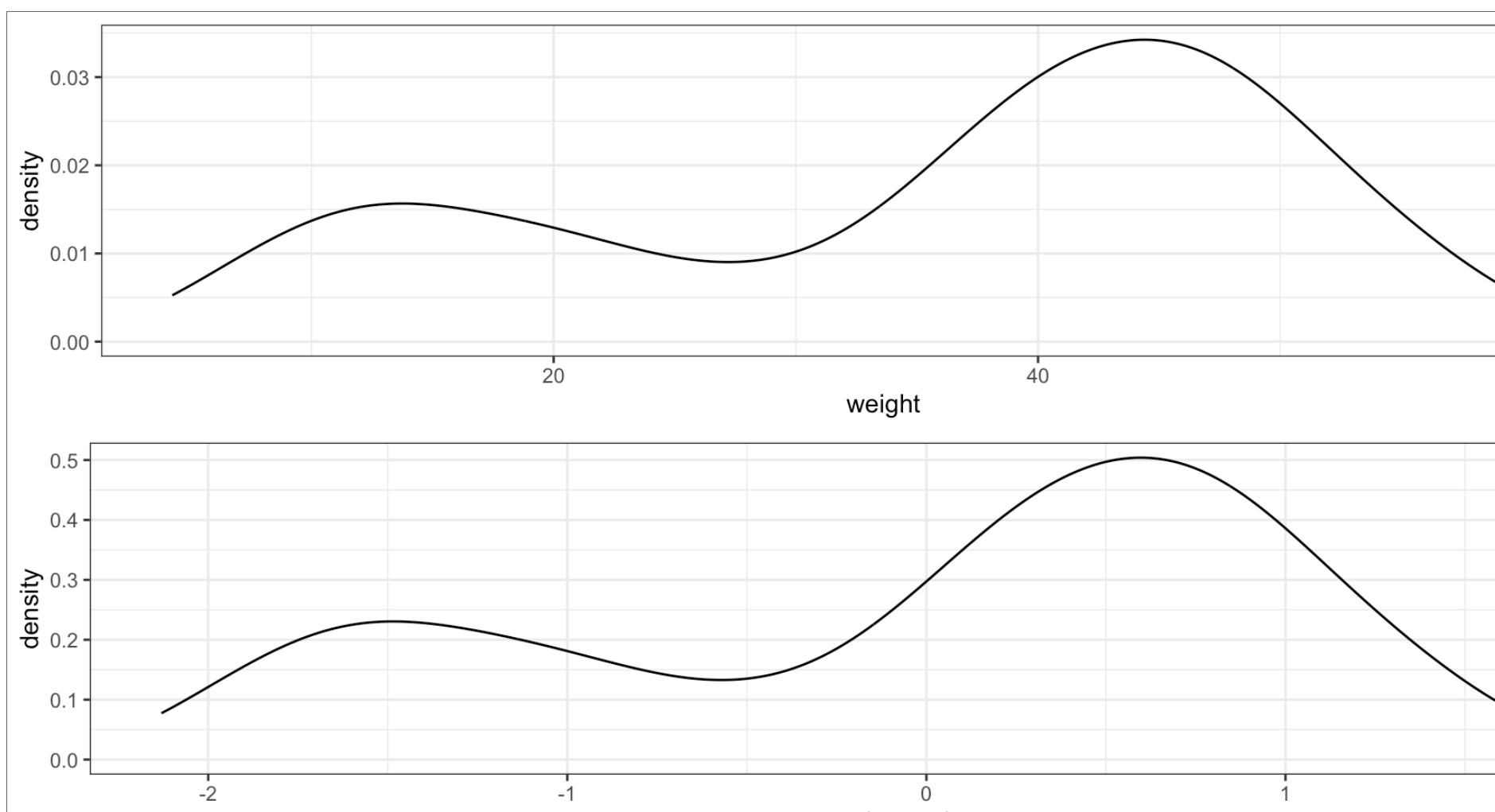
Wait what is an exponential distribution?



Scaling variables

Rescaling variables: $\frac{x_i - \bar{x}}{sd(x)}$ doesn't change the shape of a variable's distribution.

Priors are easier to define and models easier to fit. It's a good default move.



scale(weight)

Prior simulation

```
1 prior_sims <- data.frame(  
2   n_sim = 1:1e3,  
3   a = rnorm(1e3, 0, 5), # intercept  $N(0,5)$   
4   b1 = rlnorm(1e3, 0, 1), # linear term  $\log N(0,1)$   
5   b2 = rnorm(1e3, 0, 1), # quadratic term  $N(0,1)$   
6   sigma = rexp(1e3, 1)) # sigma  $\exp(1)$ 
```

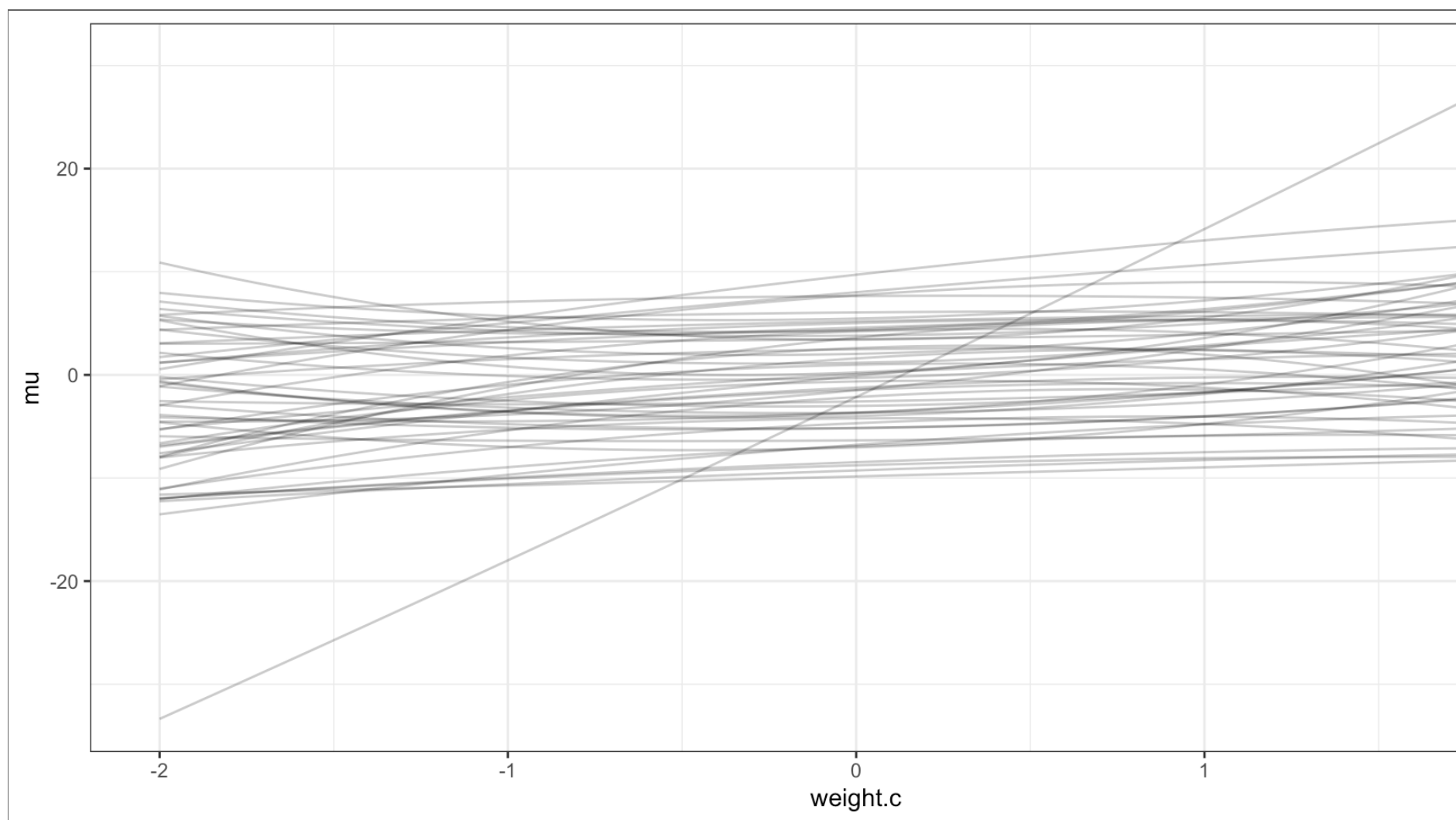
Prior simulation, linear predictor

```
1 pred_dat <- expand_grid(  
2   weight.c = seq(-2, 2, length.out=100), # define centered weight grid  
3   n_sim = 1:1e3) |> # for each simulation  
4   left_join(prior_sims) # attach prior simulations  
5 # compute mu, simulate height  
6 pred_dat <- pred_dat |>  
7   mutate(mu = a + b1*weight.c + b2*weight.c^2)  
8 # what's inside?  
9 head(pred_dat)
```

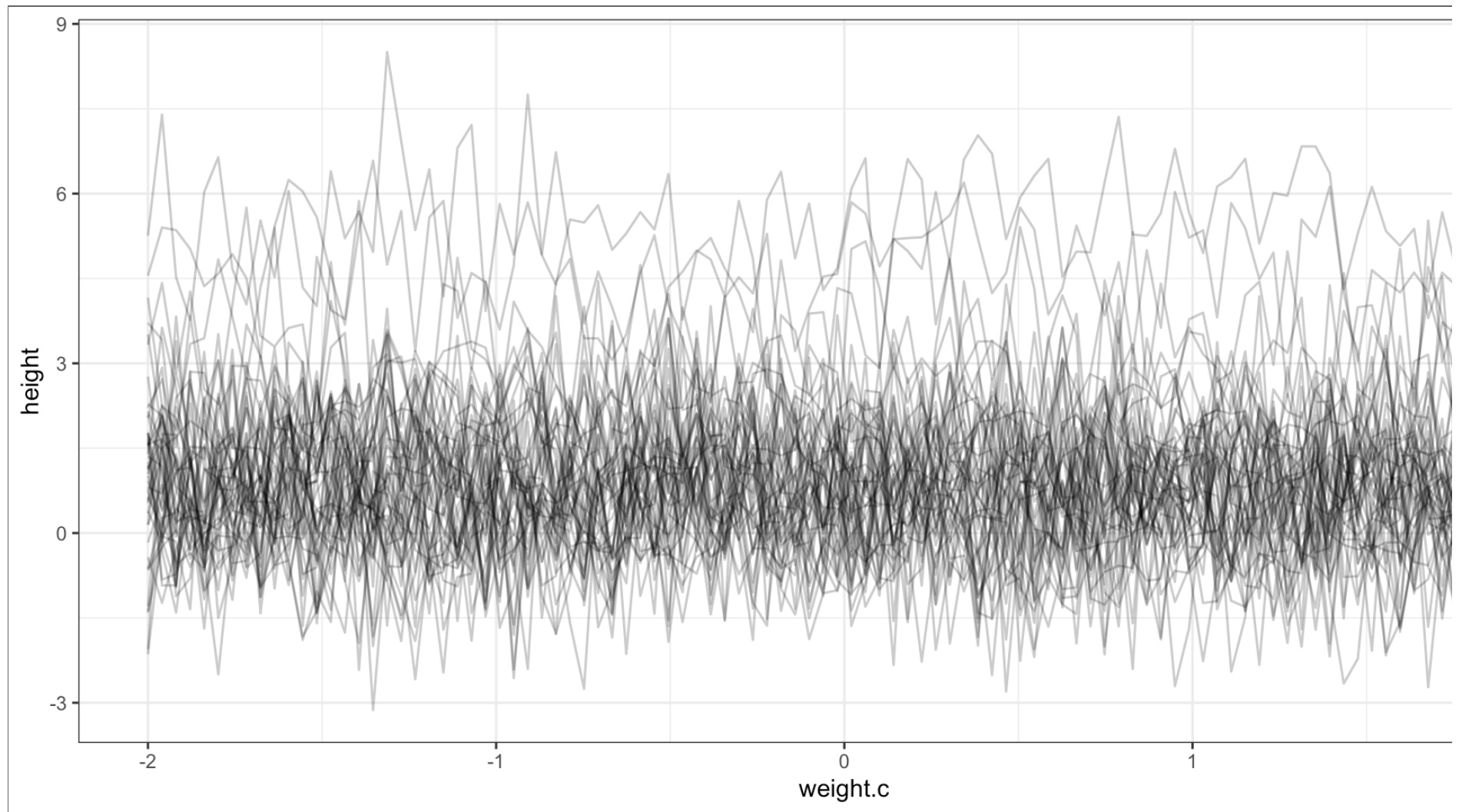
A tibble: 6 × 7

	weight.c	n_sim	a	b1	b2	sigma	mu
	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	-2	1	12.5	5.49	0.694	0.935	4.25
2	-2	2	-4.49	2.28	1.61	0.361	-2.62
3	-2	3	-4.02	0.778	-1.17	0.848	-10.3
4	-2	4	-2.56	1.42	0.305	1.26	-4.16
5	-2	5	2.70	2.47	0.570	0.914	0.0321
6	-2	6	-3.70	1.40	0.595	0.559	-4.11

Prior simulation, linear predictor (50 sims)



Prior predictive simulation



Fitting the model

```
1 d2<-d2 |>
2   mutate(height.s = scale(height),
3           weight.s = scale(weight),
4           weight.s2 = scale(weight)^2)
5
6 m_quadratic_priors <- c(
7   set_prior("normal(0,5)", class = "Intercept"),
8   set_prior("lognormal(0, 1)", class = "b", coef = "weight.s"), # this will throw a warning for ze
9   set_prior("normal(0,1)", class = "b", coef = "weight.s2"),
10  set_prior("exponential(1)", class = "sigma"))
11
12 m_quadratic <- brm(height.s ~ weight.s + weight.s2,
13                   prior = m_quadratic_priors,
14                   data = d2,
15                   silent = 2)
```

Summary

```
1 summary(m_quadratic)
```

```
Family: gaussian
Links: mu = identity
Formula: height.s ~ weight.s + weight.s2
Data: d2 (Number of observations: 544)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000
```

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	0.30	0.01	0.28	0.33	1.00	3710	3032
weight.s	0.78	0.01	0.75	0.80	1.00	3292	3095
weight.s2	-0.30	0.01	-0.33	-0.28	1.00	3377	2937

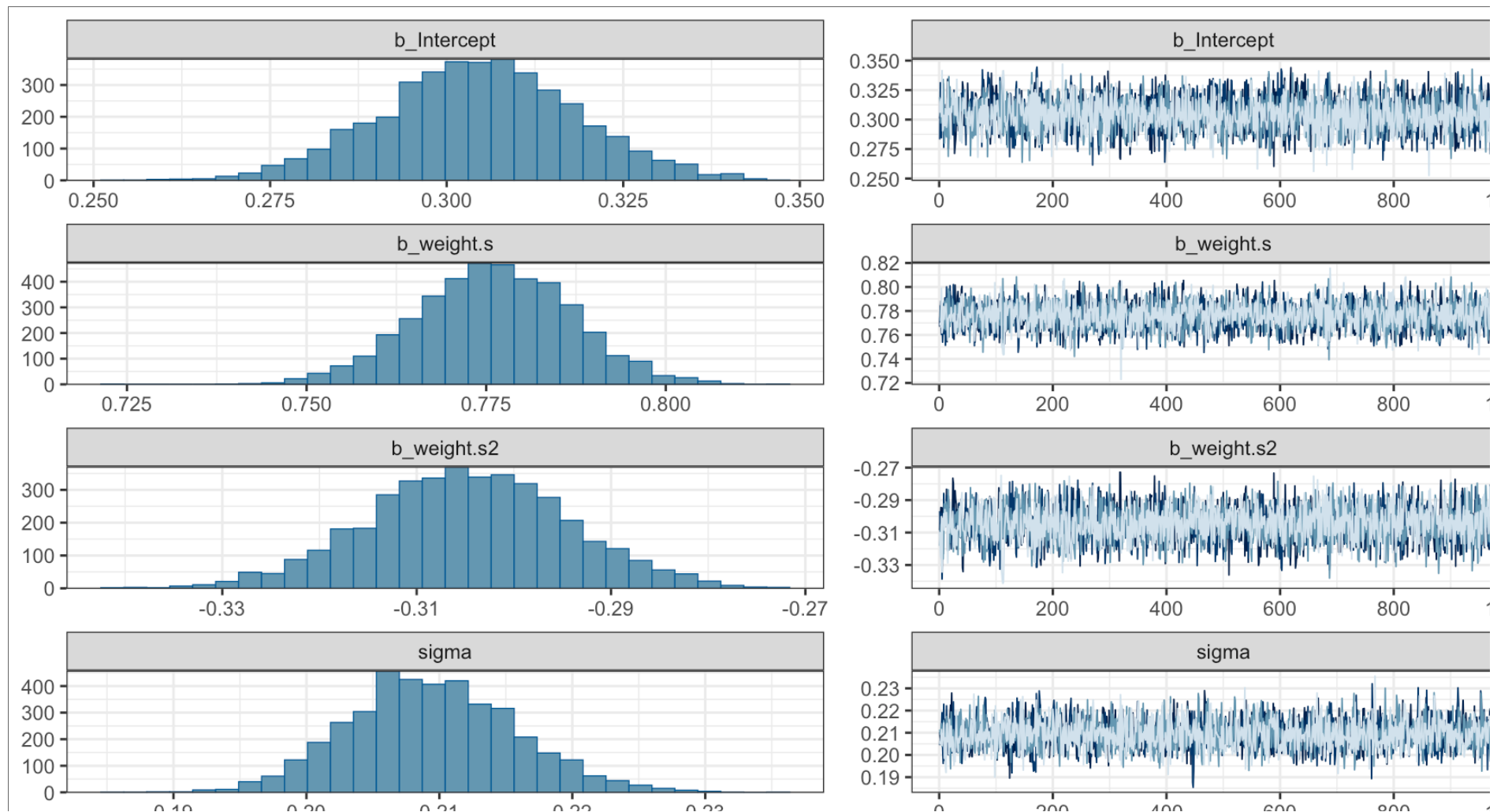
Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	0.21	0.01	0.20	0.22	1.00	3860	3166

Draws were sampled using `sampling(NUTS)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat` = 1).

Evaluating the posterior for each parameter

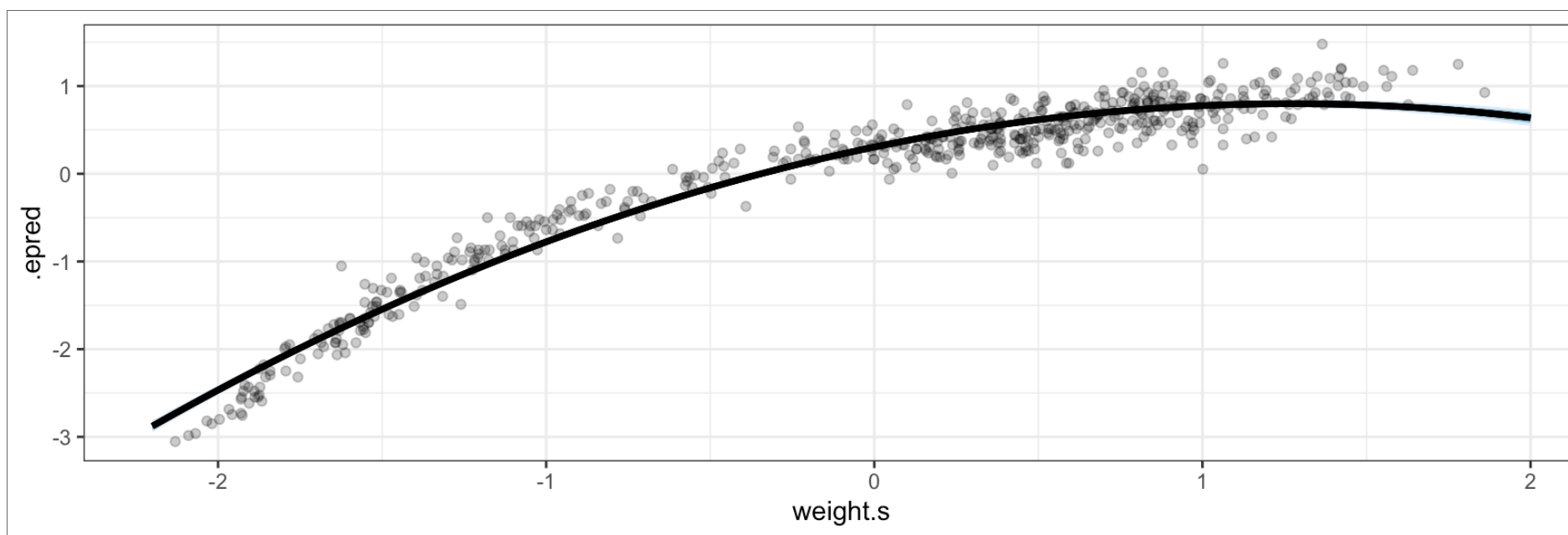
- 1 # easy mode
- 2 `plot(m_quadratic)`





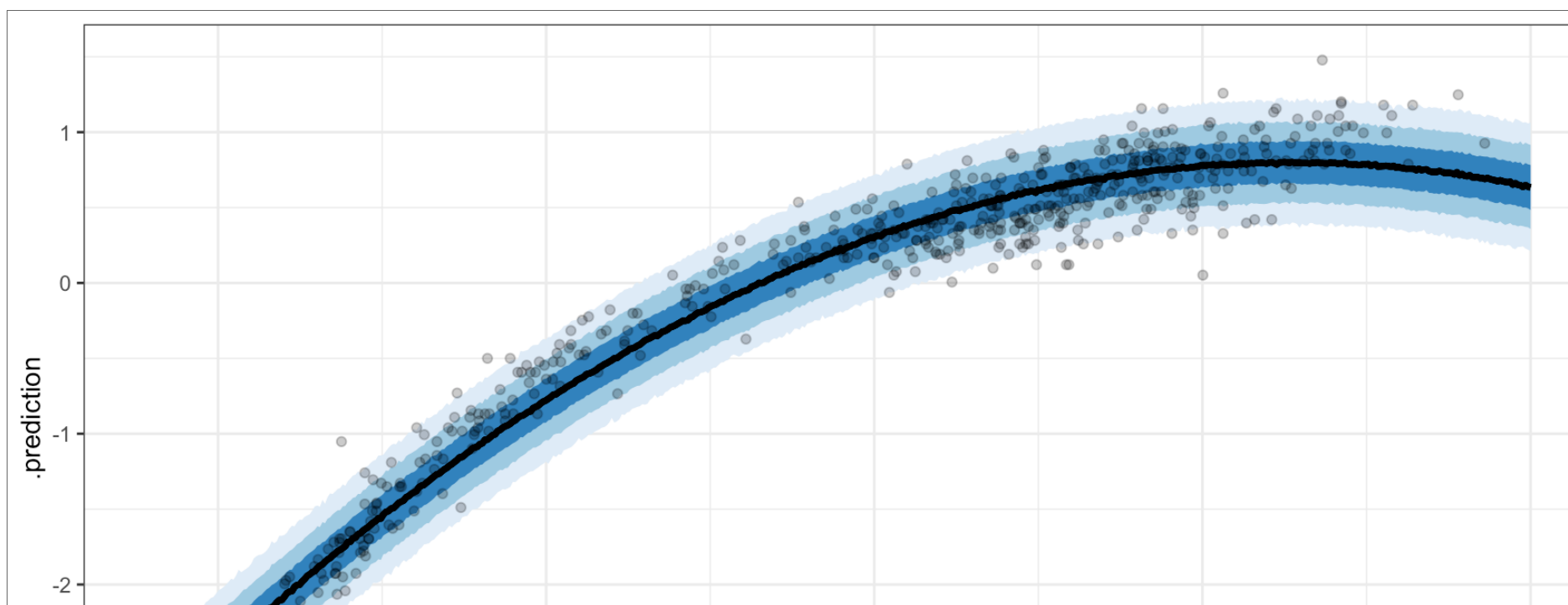
Visualize the fit: mu

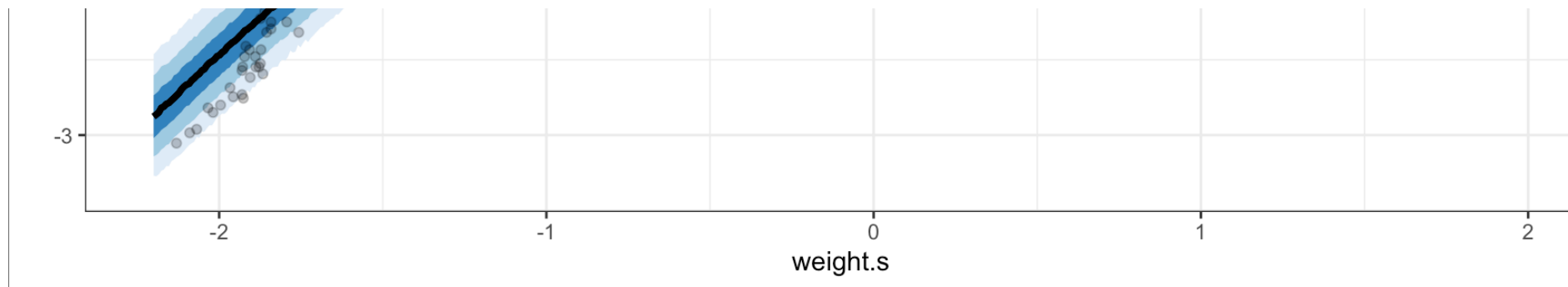
```
1 sim_dat<-data.frame(weight.s = seq(-2.2, 2, length.out = nrow(d2))) |>  
2   mutate(weight.s2 = weight.s^2) |>  
3   add_epred_draws(m_quadratic)  
4  
5 ggplot(d2,  
6   aes(x = weight.s, y = height.s)) +  
7   stat_lineribbon(data = sim_dat,  
8     aes(x = weight.s, y = .epred)) +  
9   geom_point(alpha = 0.25) +  
10  scale_fill_brewer()
```



Visualize posterior predictive fit

```
1 sim_dat<-data.frame(weight.s = seq(-2.2, 2, length.out = nrow(d2))) |>  
2   mutate(weight.s2 = weight.s^2) |>  
3   add_predicted_draws(m_quadratic)  
4  
5 ggplot(d2,  
6       aes(x = weight.s, y = height.s)) +  
7   stat_lineribbon(data = sim_dat,  
8               aes(x = weight.s, y = .prediction)) +  
9   geom_point(alpha = 0.25) +  
10  scale_fill_brewer()
```

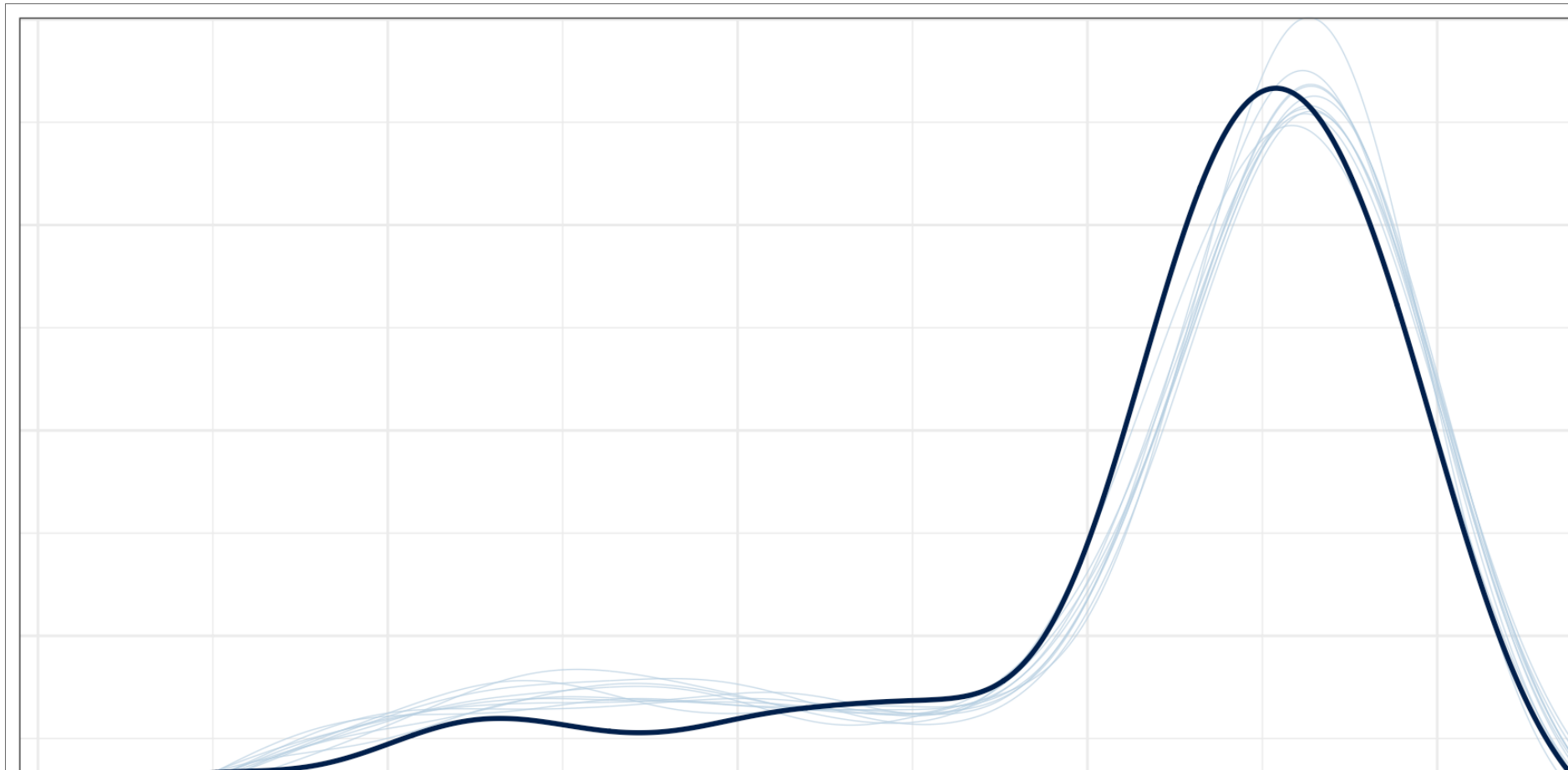


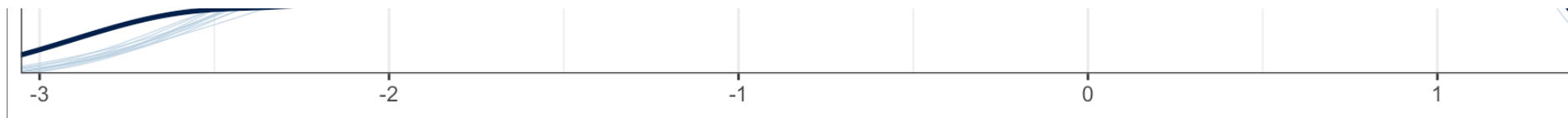


Posterior predictive check, easy mode

Could probably use the extra detail a cubic would get us, doesn't look great for children

```
1 pp_check(m_quadratic)
```

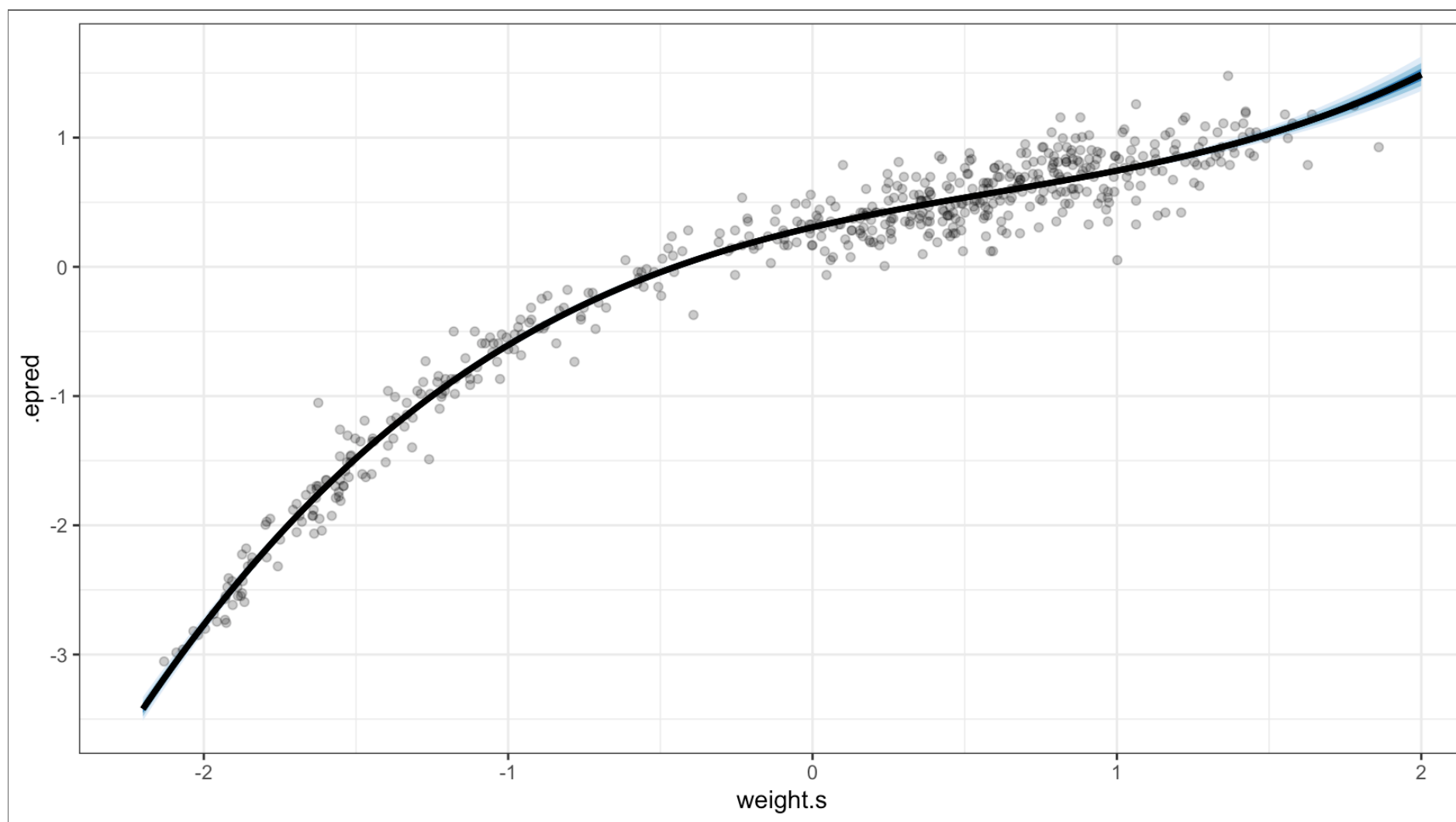




Fit a cubic model

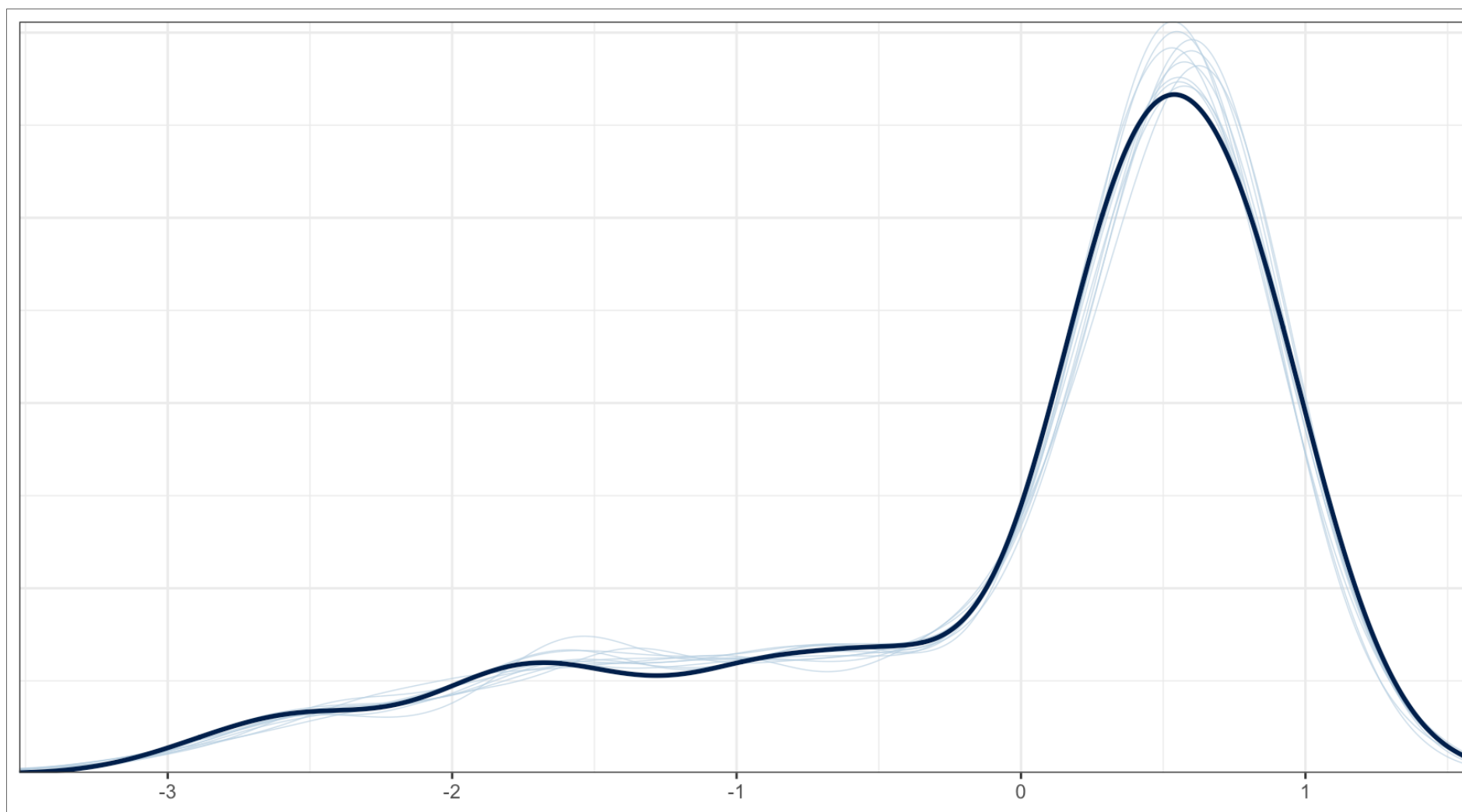
```
1 d2<-d2 |>
2   mutate(height.s = scale(height),
3           weight.s = scale(weight),
4           weight.s2 = scale(weight)^2,
5           weight.s3 = scale(weight)^3)
6
7 m_cubic <- c(
8   set_prior("normal(0,5", class = "Intercept"),
9   set_prior("lognormal(0, 1)", class = "b", coef = "weight.s"), # this will throw a warning for ze
10  set_prior("normal(0,1)", class = "b", coef = "weight.s2"),
11  set_prior("normal(0,1)", class = "b", coef = "weight.s3"),
12  set_prior("exponential(1)", class = "sigma"))
13
14 m_cubic <- brm(height.s ~ weight.s + weight.s2 + weight.s3,
15               prior = m_quadratic_priors,
16               data = d2)
```

Visualize posterior for μ



Posterior predictive check

```
1 pp_check(m_cubic)
```



GAMs and splines

Motivation

Sometimes it's reasonable to use polynomials, logs, or other non-linear transforms when we have good theory on the functional form of a relationship

$$E(\text{crime}_i) = \beta_1 \cdot \text{age}_i + \beta_2 \cdot \text{age}_i^2$$

But sometimes we don't have much theory about *how* variables relate to each other

$$E(\text{Homicide}_t) = f(\text{inflation}_t)$$

Generalized additive models (GAMs)

GAMs are very helpful when we want to *learn* about functional relationships from the data.

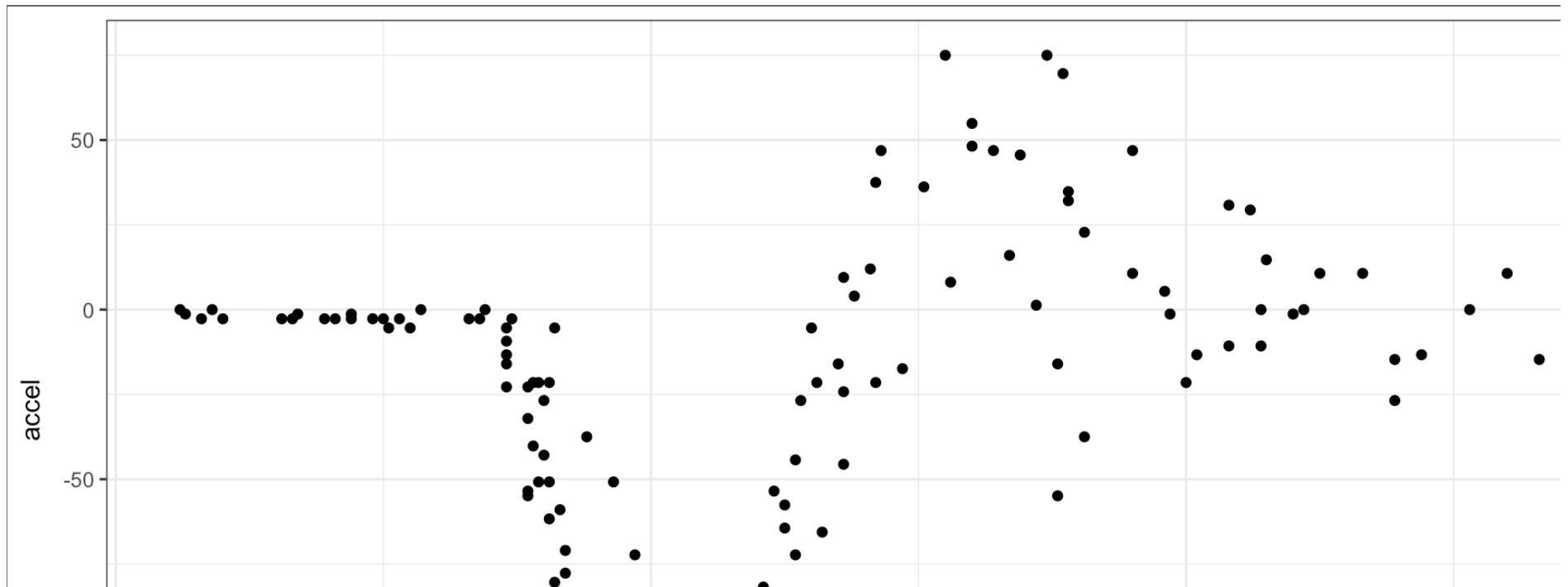
We change our linear model $E(y) = \alpha + \beta x$ into a more flexible model

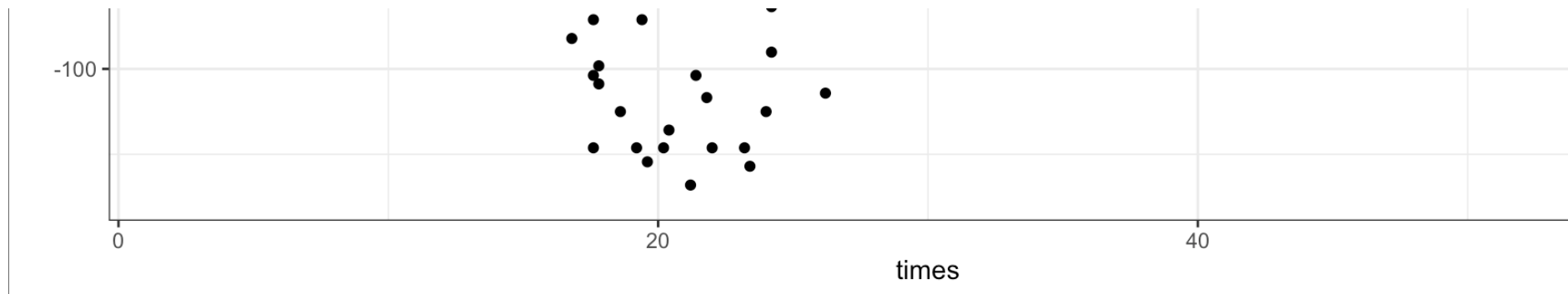
$$E(y) = \alpha + f(x)$$

Where we will estimate f when we fit the model

A simple example: Time and acceleration for motorcycle driver's head after crash (ouch!)

```
1 data_motor <- MASS::mcycle |>  
2   tibble() |>  
3   unique()  
4  
5 # plot the data  
6 data_motor |>  
7   ggplot(aes(x = times, y = accel)) +  
8   geom_point()
```





Note: this section borrows liberally from <https://michael-franke.github.io/Bayesian-Regression/practice-sheets/10b-GAMs.html>

So what does a GAM do? Basis functions as smoothers

